

Plan de tests unitaires

PFETTracker - Plateforme de suivi des projets de fin d'études

Modèle : Test Plan Template (IEEE 829-1998 Format)

1. Identifiant du plan de test (Test Plan Identifier)

Nom	Plan de tests unitaires - PFETTracker
Niveau	Unitaire
Types de test	Structurel (règles et calculs internes)
Version	1.0
Date	23/09/2026
Auteur	Fatma Hassine
Historique des révisions	1.0 - 23/09/2026 - création du plan

2. Introduction (Introduction)

Vérifier, de façon isolée, les règles métier et les calculs critiques du système.

Périmètre : 8 exigences du catalogue. Ce plan est le niveau le plus bas : il précède le plan d'intégration, le plan système et le plan d'acceptation.

Références :

- Cahier des charges de PFETTracker (version 2.0).
- Catalogue des 68 exigences fonctionnelles du rapport.
- Modèle IEEE 829-1998 « Test Plan Template » (Software Quality Engineering, version 7.0, 2001), diffusé par l'université CSUN.
- Kiwi TCMS : produit PFETTracker, ses cas de test et ses plans.

3. Éléments à tester (Test Items)

Règles et calculs internes des trois modules :

- Rotation du jeton JWT (exigence n°3)
- Limitation de débit des tentatives (n°7)
- Cycle de statuts d'un sprint (n°41)
- Statuts d'un jalon (n°43)
- Cycle de statuts d'une tâche (n°45)
- Priorité d'une tâche (n°48)
- Lien de secours Google Meet (n°63)

- Préférences de notification (n°67)

Version testée : [numéro de version à compléter]

4. Fonctionnalités à tester *(Features To Be Tested)*

Liste vue depuis l'utilisateur, avec le niveau de risque de chaque exigence : H = élevé, M = moyen, L = faible. Il correspond à la priorité P1, P2 ou P3 du cas de test dans Kiwi TCMS.

N°	Exigence	Module	Priorité	Risque
3	Le système doit permettre de rafraîchir un jeton JWT expiré via un refresh token, avec rotation de ce dernier à chaque rafraîchissement	Module 1	P1	Élevé (H)
7	Le système doit limiter le nombre de tentatives de connexion (10/min) et d'inscription (5/min) par IP pour prévenir la force brute	Module 1	P1	Élevé (H)
41	Un sprint doit suivre un cycle de statuts : planifié → en cours → bilan soumis → validé → clôturé	Module 2	P2	Moyen (M)
43	Un jalon doit suivre un statut parmi : non démarré, en cours, terminé	Module 2	P3	Faible (L)
45	Le système doit permettre de changer le statut d'une tâche selon le cycle : non démarrée → en cours → soumise → validée / à corriger / annulée	Module 2	P1	Élevé (H)
48	Chaque tâche doit pouvoir recevoir une priorité : basse, normale, haute, critique	Module 2	P3	Faible (L)
63	En cas d'échec de génération d'un lien Meet (accès Google non autorisé), le système doit basculer automatiquement sur un lien de secours	Module 3	P1	Élevé (H)
67	Le système doit permettre à chaque utilisateur de configurer ses préférences de notification par canal (email, in-app, websocket) et par type d'événement	Module 3	P2	Moyen (M)

5. Fonctionnalités à ne pas tester *(Features Not To Be Tested)*

- Interactions entre modules et services externes réels (plan d'intégration).
- Parcours complets de bout en bout (plan système).
- Les autres exigences du catalogue.

6. Approche *(Approach)*

Stratégie : Chaque règle est vérifiée séparément du reste du système. En pratique, ces vérifications s'automatisent avec JUnit 5 et Mockito côté backend ; Kiwi TCMS sert à les référencer et à suivre leur résultat.

Outils :

- JUnit 5
- Mockito
- Kiwi TCMS

Métriques collectées :

- Taux de cas passés.

Régression : Le plan est rejoué après toute modification d'une règle testée.

Configurations testées : une seule configuration, l'environnement local.

7. Critères de réussite / d'échec *(Item Pass/Fail Criteria)*

Le plan est considéré comme réussi lorsque :

- 100 % des cas du plan passés.
- Toute anomalie est corrigée puis re-testée.
- Aucune anomalie ouverte n'est acceptée sur les règles de sécurité (exigences n°3 et n°7).

Une anomalie (defect) est un défaut pouvant provoquer une défaillance ; une défaillance (failure) est son effet visible par l'utilisateur.

8. Critères de suspension et de reprise *(Suspension Criteria and Resumption Requirements)*

Suspension : La règle testée ne peut pas être isolée, ou le code ne compile plus.

Reprise : Après correction, en rejouant les cas déjà passés.

9. Livrables *(Test Deliverables)*

- Plan de test : le présent document.
- Spécifications des cas de test : les 8 cas rattachés à ce plan dans Kiwi TCMS.
- Journaux d'exécution : résultats enregistrés dans Kiwi TCMS.
- Rapports d'incident : anomalies détectées, avec leur gravité.
- Rapport de synthèse des tests : taux de réussite par module et par priorité.
- Données de test.

Le logiciel testé n'est pas un livrable de ce plan : il est fourni par le développement.

10. Tâches de test *(Test Tasks)*

1. Préparer les données de test et les comptes de chaque rôle.
2. Exécuter les cas dans l'ordre des priorités (P1, puis P2, puis P3) et enregistrer chaque

résultat dans Kiwi TCMS.

3. Déclarer chaque anomalie avec sa gravité.
4. Rejouer les cas corrigés (re-test).
5. Rédiger le rapport de synthèse.

11. Besoins en environnement *(Environmental Needs)*

- Application PFETracker : backend Spring Boot 3.2 (Java 17), frontend React, base de données MySQL, déployée en local.
- Kiwi TCMS (Docker) : suivi des cas de test, des plans et des résultats.
- Comptes de test : un par rôle (étudiant, encadrant, chef de département, directeur des stages, service des stages, administrateur).
- Données de test : créées avant chaque campagne (départements, affectations, PFE, tâches).
- Sécurité : les comptes de test sont distincts des comptes réels.
- JUnit 5 et Mockito pour l'automatisation des règles.

12. Responsabilités *(Responsibilities)*

Chaque module a un testeur responsable, qui exécute ses cas et enregistre les résultats dans Kiwi TCMS.

Module	Testeur responsable	Cas de ce plan
Module 1 - Administration	[nom à compléter]	2
Module 2 - Suivi du PFE	[nom à compléter]	4
Module 3 - Communication	[nom à compléter]	2

La décision d'arrêter ou de poursuivre le plan revient à l'encadrant.

13. Besoins en personnel et en formation *(Staffing and Training Needs)*

Les trois testeurs doivent maîtriser :

- Kiwi TCMS : création de cas, de plans et d'exécutions.
- JUnit 5 et Mockito.

14. Planning *(Schedule)*

Jalon	Date
Livraison de la version à tester	[date à compléter]
Début des tests	Le lendemain de la livraison
Fin des tests	[date à compléter]

Jalon	Date
Rapport de synthèse	[date à compléter]

Conformément au modèle, les dates de test sont liées aux dates de livraison des versions, et non à des dates fixes. Si la livraison est en retard, le test démarre à la date réelle de livraison et l'encadrant est prévenu.

15. Risques et plans de secours *(Risks and Contingencies)*

Risque	Parade
Manque de disponibilité des testeurs	Répartition des cas par module entre les trois testeurs ; exécution d'abord des cas P1.
Application ou données de test indisponibles	Redéployer l'application et recréer les comptes et données de test avant chaque campagne.
Livraison tardive d'une version	Le test démarre à la date réelle de livraison ; le retard est signalé à l'encadrant.
Changement des exigences	Adapter les cas touchés par le changement, puis rejouer le plan d'acceptation.
Règle mal isolée (dépendance cachée)	La simuler ou la remplacer par une donnée de test.

16. Approbations *(Approvals)*

Le plan est considéré comme terminé lorsqu'il est approuvé par :

Rôle	Nom	Date	Signature
Encadrant du projet	[nom à compléter]		
Responsable de l'équipe de test	[nom à compléter]		